LAB SHEET 04 (FULL STACK)

Name: Vansh

CUID:24250033

Sec:B

RollNo.:64

Task 1 – Custom Modules

sum.js

```
function sum(a, b) {
    return a + b;
}
module.exports = sum;
```

even-odd.js

```
function checkEvenOdd(number) {
    if (number % 2 === 0) {
        return "Even";
    }
    return "Odd";
}
module.exports = checkEvenOdd;
```

app1.js

```
const sum = require("./sum");
const checkEvenOdd = require("./even-odd");

const a = 10;
const b = 20;
const number = 7;

console.log("Sum =", sum(a, b));
console.log(number, "is", checkEvenOdd(number));
```

Task 2 – File System CRUD

file-ops.js

```
const fs = require("fs");
const fileName = "student.txt";
const data = `
Name: Vansh
Roll Number: 64
Course: Full Stack Web Development
`;
fs.writeFile(fileName, data, (err) => {
    if (err) {
        console.log("Error creating file:", err);
        return;
    }
```

```
console.log("File created successfully.");

fs.appendFile(
    fileName,
    "College: ABC University\n",
    (err) => {
        if (err) {
            console.log("Error appending file:", err);
            return;
        }

        console.log("Data appended successfully.");

        fs.readFile(fileName, "utf8", (err, data) => {
            if (err) {
                console.log("Error reading file:", err);
                return;
            }

            console.log("\nFile Content:");
            console.log(data);

            fs.rename(fileName, "profile.txt", (err) => {
                if (err) {
```

```
                console.log("Error renaming file:", err);
                return;
            }

            console.log("File renamed to profile.txt");
        });
    });
  }
);
});
```

Task 3 – Directory Operations

dir-ops.js

```
const fs = require("fs");

const folder = "uploads";

if (!fs.existsSync(folder)) {
  fs.mkdirSync(folder);
  console.log("uploads directory created.");
}

for (let i = 1; i <= 3; i++) {
  const filePath = `${folder}/file${i}.txt`;
```

```
    if (!fs.existsSync(filePath)) {
        fs.writeFileSync(filePath, "");
    }
}

console.log("Files in uploads directory:");
console.log(fs.readdirSync(folder));

const deleteFile = `${folder}/file2.txt`;

if (fs.existsSync(deleteFile)) {
    fs.unlinkSync(deleteFile);
    console.log("file2.txt deleted.");
}

console.log("Files after deletion:");
console.log(fs.readdirSync(folder));
```

Task 4 – Path Module

path-demo.js

```
const path = require("path");

const filePath =
    "C:\\Users\\Student\\Documents\\project\\app.js";
```

```
console.log("Complete Path:");
console.log(filePath);

console.log("\nDirectory Name:");
console.log(path.dirname(filePath));

console.log("\nFile Name:");
console.log(path.basename(filePath));

console.log("\nFile Extension:");
console.log(path.extname(filePath));

console.log("\nAbsolute Path:");
console.log(path.resolve(filePath));
```

Task 5 – OS Module Report

sys-report.js

```
const os = require("os");

console.log("===== SYSTEM REPORT =====");

console.log("Operating System:", os.platform());

console.log("CPU Architecture:", os.arch());
```

```
console.log(
   "Total Memory:",
   (os.totalmem() / (1024 ** 3)).toFixed(2),
   "GB"
);

console.log(
   "Free Memory:",
   (os.freemem() / (1024 ** 3)).toFixed(2),
   "GB"
);

console.log("Number of CPU Cores:", os.cpus().length);

console.log(
   "System Uptime:",
   (os.uptime() / 60).toFixed(2),
   "minutes"
);
```

Task 6 – Events / EventEmitter

order-events.js

```
const EventEmitter = require("events");

const orderSystem = new EventEmitter();
```

```
orderSystem.on("orderPlaced", (order) => {
    console.log("\n===== ORDER RECEIPT =====");
    console.log("Order ID:", order.id);
    console.log("Item:", order.item);
    console.log("Amount: ₹" + order.amount);
    console.log("Order placed successfully!");
});

const orders = [
    {
        id: 101,
        item: "Laptop",
        amount: 55000
    },
    {
        id: 102,
        item: "Smartphone",
        amount: 25000
    },
    {
        id: 103,
        item: "Headphones",
        amount: 2500
    }
```

```
];

orders.forEach((order) => {
   orderSystem.emit("orderPlaced", order);
});
```

Task 7 – HTTP Server

server.js

```
const http = require("http");

const server = http.createServer((req, res) => {

   if (req.url === "/" && req.method === "GET") {

      res.writeHead(200, {
         "Content-Type": "text/html"
      });

      res.end(`
         <h1>Welcome to Node.js Server</h1>
         <p>Home Page</p>
      `);

   } else if (req.url === "/students" && req.method === "GET") {
```

```
    res.writeHead(200, {
        "Content-Type": "application/json"
    });

    const students = [
        { id: 1, name: "Shubh" },
        { id: 2, name: "Rahul" },
        { id: 3, name: "Aman" }
    ];

    res.end(JSON.stringify(students));

} else if (
    req.url.startsWith("/students/") &&
    req.method === "GET"
) {

    const id = req.url.split("/")[2];

    res.writeHead(200, {
        "Content-Type": "application/json"
    });

    res.end(JSON.stringify({
        id: id,
```

```
            name: "Student " + id
        }));

    } else {

        res.writeHead(404, {
            "Content-Type": "text/plain"
        });

        res.end("404- Page Not Found");
    }
});

server.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Task 8 – Query String Parsing

server.js

```
const http = require("http");
const url = require("url");

const server = http.createServer((req, res) => {

    const parsedUrl = url.parse(req.url, true);
```

```
    if (parsedUrl.pathname === "/search") {

        const keyword = parsedUrl.query.keyword;

        res.writeHead(200, {
            "Content-Type": "text/html"
        });

        res.end(`
            <h1>Search Page</h1>
            <p>Keyword: ${keyword || "No keyword provided"}</p>
        `);

    } else {

        res.writeHead(404, {
            "Content-Type": "text/plain"
        });

        res.end("404- Page Not Found");
    }
});

server.listen(3001, () => {
    console.log("Server running at http://localhost:3001");
```

```
});
```

Task 9 – Streams

stream-demo.js

```
const fs = require("fs");

const fileName = "large-file.txt";

let data = "";

for (let i = 1; i <= 50; i++) {
    data += `This is line number ${i}\n`;
}

fs.writeFileSync(fileName, data);

console.log("Large file created.");

const stream = fs.createReadStream(fileName, {
    encoding: "utf8"
});

stream.on("data", (chunk) => {
    console.log("Received chunk");
    console.log("Chunk size:", chunk.length);
```

```
});

stream.on("end", () => {
   console.log("Finished reading the file.");
});

stream.on("error", (err) => {
   console.log("Error:", err.message);
});
```

Task 10 – npm and Nanoid

package.json

```
{
   "name": "nanoid-example",
   "version": "1.0.0",
   "main": "id-gen.js",
   "dependencies": {
      "nanoid": "^3.3.8"
   }
}
```

id-gen.js

```
const { nanoid } = require("nanoid");

console.log("===== GENERATED IDs =====");
```

```
for (let i = 1; i <= 5; i++) {
  console.log(`${i}. ${nanoid()}`);
}
```

Task 11 – Command Line Calculator

calc-cli.js

```
const args = process.argv.slice(2);

if (args.length !== 3) {
  console.log(
    "Usage: node calc-cli.js <number1> <operator> <number2>"
  );

  process.exit();
}

const num1 = Number(args[0]);
const operator = args[1];
const num2 = Number(args[2]);

if (isNaN(num1) || isNaN(num2)) {
  console.log("Please enter valid numbers.");
  process.exit();
}
```

```
let result;
switch (operator) {
    case "+":
        result = num1 + num2;
        break;

    case "-":
        result = num1- num2;
        break;

    case "*":
        result = num1 * num2;
        break;
    case "/":
        if (num2 === 0) {
            console.log("Cannot divide by zero.");
            process.exit();
        }
        result = num1 / num2;
        break;
    default:
        console.log("Invalid operator.");
        process.exit();
}
```

```
console.log("Result:", result);
```

Task 12 – Environment Variables

env-demo.js

```
const http = require("http");

const PORT = process.env.PORT || 4000;

const server = http.createServer((req, res) => {

    res.writeHead(200, {
        "Content-Type": "text/html"
    });

    res.end(`
        <h1>Environment Variable Demo</h1>
        <p>Server is running on port: ${PORT}</p>
    `);
});

server.listen(PORT, () => {
    console.log(
        `Server running at http://localhost:${PORT}`
    );
});
```